Aufgabe 1 Abstrakte Klassen

Querschnittswerte

Querschnitte sollen einheitlich behandelt werden können.

1. Abstrakte Klasse `CrossSection` mit abstrakten Methoden `Area()` und `SecondMoment()` (Flächenträgheitsmoment I_y).
2. Unterklassen `Rectangle` (Breite b, Höhe h) und `Circle` (Durchmesser d).

$$A = bh, \quad I_y = \frac{bh^3}{12} \quad \text{bzw.} \quad A = \frac{\pi d^2}{4}, \quad I_y = \frac{\pi d^4}{64}$$

3. Testen Sie beide Klassen in einem kurzen Programm.

> **Erweiterung:** Speichern Sie mehrere Querschnitte in einer `List<CrossSection>` und geben Sie den Querschnitt mit dem größten I_y aus.

Aufgabe 2 Abstrakte Klassen

Lasten auf einem Träger

Auf einem Träger wirken verschiedene Lastarten. Gesucht ist jeweils die Resultierende.

1. Abstrakte Klasse `Load` mit abstrakter Methode `Resultant()`.
2. Unterklassen:
   - `PointLoad` mit Kraft $F \to R = F$
   - `UniformLoad` mit q und Länge $l \to R = ql$
   - `TriangularLoad` mit $q_{\max}$ und $l \to R = q_{\max} l/2$
3. Speichern Sie einige Lasten in einer Liste und berechnen Sie die Summe aller Resultierenden.

> **Erweiterung:** Ergänzen Sie in jeder Unterklasse `ToString()`, sodass die Last mit Typ und Werten ausgegeben wird.

Aufgabe 3 Abstrakte Klassen

Eigengewicht von Bauteilen

Das Eigengewicht eines Bauteils ist $G = \gamma \cdot V$ mit der Wichte γ in kN/m³.

1. Abstrakte Klasse `Member` mit Attribut `UnitWeight` (γ), abstrakter Methode `Volume()` und **nicht** abstrakter Methode `Weight()`.
2. Unterklassen `RectangularColumn` (Breite, Tiefe, Höhe) und `CircularColumn` (Durchmesser, Höhe).
3. Berechnen Sie das Gewicht einer Stahlbetonstütze 30 × 30 cm, Höhe 3.0 m ($\gamma = 25$ kN/m³).

> **Erweiterung:** Warum muss `Weight()` nur einmal programmiert werden? Erklären Sie, welche `Volume()`-Methode jeweils aufgerufen wird.

Aufgabe 4 Abstrakte Klassen

Wand mit Öffnungen

Für eine Wand soll die Nettofläche (ohne Öffnungen) berechnet werden.

1. Abstrakte Klasse `Opening` mit abstrakter Methode `Area()`.
2. Unterklassen `Window` (Breite, Höhe) und `RoundWindow` (Durchmesser).
3. Klasse `Wall` mit Länge, Höhe und einer `List<Opening>`, Methoden `AddOpening(Opening o)` und `NetArea()`.
4. Testen Sie mit einer Wand 6.0 × 2.75 m mit zwei Fenstern und einem runden Fenster.

> **Erweiterung:** Ergänzen Sie eine Unterklasse `Door`. Welche Stellen im Programm müssen Sie dafür ändern?

Aufgabe 5 Abstrakte Klassen

Statische Bestimmtheit

Ein Einfeldträger ist statisch bestimmt, wenn die Lager zusammen genau drei Auflagerreaktionen haben.

1. Abstrakte Klasse `Support` mit abstrakter Methode `Reactions()`.
2. Unterklassen `FixedSupport` (3), `PinnedSupport` (2) und `RollerSupport` (1).
3. Klasse `Beam` mit einer Liste von Lagern und Methode `Degree()`, die $n = \sum r - 3$ zurückgibt.
4. Methode `Classify()` gibt „kinematisch“ ($n < 0$), „statisch bestimmt“ ($n = 0$) oder „n-fach statisch unbestimmt“ zurück.

Aufgabe 6 Abstrakte Klassen

Leistungsverzeichnis

Ein Angebot besteht aus Positionen, deren Preis unterschiedlich ermittelt wird.

1. Abstrakte Klasse `Item` mit Attribut `Description` und abstrakter Methode `Price()`.
2. Unterklassen:
   - `QuantityItem`: Menge × Einheitspreis (z.B. 120 m² Mauerwerk à 85 €)
   - `LumpSumItem`: Pauschalpreis (z.B. Baustelleneinrichtung 4500 €)
3. Klasse `Offer` mit Liste von Positionen und Methode `Total()`.

> **Erweiterung:** Überschreiben Sie `ToString()` in den Unterklassen und geben Sie das Angebot zeilenweise mit Gesamtsumme aus.

Aufgabe 7 — Abstrakte Klassen

Brücke mit Lastbegrenzung

Eine Brücke darf nur befahren werden, solange die zulässige Gesamtlast nicht überschritten wird.

1. Abstrakte Klasse `Vehicle` mit abstrakter Methode `Weight()` (in t).
2. Unterklassen `Car` (immer 1.5 t) und `Truck` (Anzahl Achsen × Achslast).
3. Klasse `Bridge` mit `MaxLoad` und einer Liste der Fahrzeuge auf der Brücke.
4. Methode `TryEnter(Vehicle v)`: Fahrzeug nur hinzufügen, wenn die Last danach zulässig ist; Rückgabewert `true` oder `false`.

> **Erweiterung:** Methode `Leave(Vehicle v)` und Ausgabe der aktuellen Auslastung in Prozent.

Aufgabe 8 — Klassen

Vektoren in der Ebene

Erstellen Sie eine Klasse `Vector2D` mit den Attributen `X` und `Y` und den Methoden

- `Length()` – Betrag des Vektors
- `Add(Vector2D v)` – Summe als neuer Vektor
- `Scale(double s)` – mit s multiplizierter Vektor
- `Dot(Vector2D v)` – Skalarprodukt $x_1x_2 + y_1y_2$

Überschreiben Sie `ToString()` und testen Sie die Klasse, z.B. mit der Resultierenden zweier Kräfte.

> **Erweiterung:** Methode `AngleTo(Vector2D v)`, die den Winkel zwischen zwei Vektoren in Grad liefert: $\cos\varphi = (a \cdot b)/(|a|\ |b|)$.

Aufgabe 9 — Klassen

Intervalle

Ein Intervall $[a, b]$ beschreibt z.B. einen zulässigen Wertebereich.

1. Klasse `Interval` mit Attributen `Min` und `Max`.
2. Methoden `Length()` und `Contains(double x)`.
3. Methode `Overlaps(Interval other)`, die prüft, ob sich zwei Intervalle überschneiden.
4. Testen Sie mit mehreren Fällen: überlappend, getrennt, eines im anderen enthalten.

> **Erweiterung:** Methode `Intersection(Interval other)`, die die Schnittmenge als neues Intervall zurückgibt – oder `null`, falls es keine gibt.

Aufgabe 10 — Klassen und Verzweigungen

Regenrückhaltebecken

Ein Becken hat ein Fassungsvermögen (m³) und einen aktuellen Füllstand (m³).

1. Klasse `Basin` mit Attributen `Capacity` und `Volume` (anfangs leer).
2. Methode `Fill(double v)`: füllt das Becken, gibt die Menge zurück, die überläuft.
3. Methode `Drain(double v)`: entleert das Becken, aber nicht unter 0; gibt die tatsächlich abgelassene Menge zurück.

> **Erweiterung:** Simulieren Sie mit einem Array von stündlichen Zuflüssen und konstantem Abfluss den Verlauf. Wie viel läuft insgesamt über?

Aufgabe 11 — Listen

Polygone

1. Klasse `Point` mit Attributen `X` und `Y`.
2. Klasse `Polygon` mit einer `List<Point>` und Methode `Add(Point p)`.
3. Methode `Perimeter()`, die den Umfang berechnet. Achtung: Das Polygon ist geschlossen.
4. Testen Sie mit einem Rechteck und einem Dreieck.

> **Erweiterung:** Methode `Area()` mit der Gaußschen Trapezformel (Index n entspricht wieder 0):
>
> $$A = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

Aufgabe 12 — Listen

Messreihe

An einem Bauwerk werden Verformungen gemessen.

1. Klasse `MeasurementSeries` mit einer `List<double>` der Messwerte.
2. Methoden `Add(double value)`, `Mean()` und `Max()`.
3. Methode `CountAbove(double limit)`, die zählt, wie viele Werte einen Grenzwert überschreiten.

Programmieren Sie `Mean()` und `CountAbove()` mit einer Schleife.

> **Erweiterung:** Methode `StandardDeviation()`:
>
> $$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2}$$

Aufgabe 13 Listen

Profilauswahl

Gesucht ist das leichteste IPE-Profil mit ausreichendem Widerstandsmoment.

1. Klasse `SteelProfile` mit `Name` und `Wy` (in cm³).
2. Klasse `ProfileCatalog` mit einer Liste von Profilen, aufsteigend nach Größe.
3. Methode `FindSuitable(double requiredWy)` gibt das erste passende Profil zurück, sonst `null`.

IPE 100: 34.2 · IPE 120: 53.0 · IPE 140: 77.3 · IPE 160: 109 · IPE 180: 146 · IPE 200: 194

> **Erweiterung:** Erforderliches W_y aus dem Moment berechnen: $W_{y,\mathrm{erf}} = M_{\mathrm{Ed}} / f_{\mathrm{yd}}$ mit $f_{\mathrm{yd}} = 23.5$ kN/cm².

Aufgabe 14 Listen

Wärmedurchgangskoeffizient

Eine Wand besteht aus Schichten mit Dicke d (m) und Wärmeleitfähigkeit λ (W/(m K)).

1. Klasse `Layer` mit Name, `Thickness`, `Lambda` und Methode `Resistance()`: $R = d/\lambda$.
2. Klasse `Wall` mit `List<Layer>` und Methode `UValue()`:

$$U = \frac{1}{R_{\mathrm{si}} + \sum R_i + R_{\mathrm{se}}} \quad \text{mit } R_{\mathrm{si}} = 0.13,\ R_{\mathrm{se}} = 0.04$$

3. Beispiel: 17.5 cm Kalksandstein ($\lambda = 0.99$), 12 cm Dämmung ($\lambda = 0.035$).

> **Erweiterung:** Erhöhen Sie die Dämmdicke in 1-cm-Schritten, bis $U \le 0.20$ ist.

Aufgabe 15 Listen

Betondruckfestigkeit

Bei Zylinderproben ($d = 150$ mm) wird die Bruchlast F in kN gemessen.

1. Klasse `Specimen` mit Bruchlast und Methode `Strength()` in N/mm².
2. Klasse `TestSeries` mit Liste von Proben, Methoden `MeanStrength()` und `MinStrength()`.
3. Methode `Satisfies(double fck)`: erfüllt, wenn Mittelwert $\ge f_{\mathrm{ck}} + 4$ und jeder Einzelwert $\ge f_{\mathrm{ck}} - 4$.
4. Testen Sie mit drei Proben: 640 kN, 610 kN, 575 kN gegen C30/37.

Aufgabe 16 Listen

Bauzeitenplan

1. Klasse `Activity` mit `Name`, `Start` und `Duration` (in Arbeitstagen) und Methode `End()`.
2. Klasse `Schedule` mit einer Liste von Vorgängen und Methode `Add(Activity a)`.
3. Methode `TotalDuration()`: Ende des Vorgangs, der zuletzt fertig wird.
4. Testen Sie z.B. mit Erdarbeiten, Fundamente, Rohbau, Dach.

> **Erweiterung:** Methode `ActiveOn(int day)`, die eine Liste aller Vorgänge zurückgibt, die an diesem Tag laufen.

Aufgabe 17 Dictionary

Eigengewicht nach Material

1. Legen Sie ein `Dictionary<string, double>` mit Wichten in kN/m³ an: Beton 25, Stahl 78.5, Holz 5.
2. Klasse `Component` mit `Name`, `Material` (Text) und `Volume` (m³).
3. Berechnen Sie für eine Liste von Bauteilen das Gesamtgewicht, indem Sie die Wichte im Dictionary nachschlagen.
4. Was passiert bei einem unbekannten Material? Fangen Sie den Fall mit `ContainsKey` ab.

> **Erweiterung:** Geben Sie das Gewicht getrennt nach Material aus (zweites Dictionary).

Aufgabe 18 Verzweigungen

Energieeffizienzklassen

1. Klasse `Building` mit `Name`, beheizter Fläche (m²) und Jahresheizwärmebedarf (kWh/a).
2. Methode `SpecificDemand()`: Bedarf pro m² und Jahr.
3. Methode `EfficiencyClass()` nach der Tabelle:

A+	A	B	C	D
< 30	< 50	< 75	< 100	≥ 100

> **Erweiterung:** Zählen Sie für eine Liste von Gebäuden mit einem `Dictionary<string, int>`, wie viele in jede Klasse fallen.

Aufgabe 19 — Dictionary und HashSet

Verkehrszählung

An einer Zählstelle werden Fahrzeuge mit Kennzeichen und Typ („Pkw“, „Lkw“, „Bus“) erfasst. Ein Fahrzeug kann mehrfach vorbeikommen.

1. Klasse `Vehicle` mit `Plate` und `Type`.
2. Klasse `Count` mit einer Liste der erfassten Fahrzeuge.
3. Methode `DistinctVehicles()`: Anzahl verschiedener Fahrzeuge (mit `HashSet<string>`).
4. Methode `CountByType()`: Anzahl Durchfahrten je Typ als `Dictionary<string, int>`.

Aufgabe 20 — Schleifen

Nullstelle mit Bisektion

Gesucht ist die Nullstelle von $f(x) = x^3 - 2x - 5$ im Intervall $[2, 3]$.

Verfahren: Mittelpunkt m des Intervalls bestimmen. Haben $f(a)$ und $f(m)$ unterschiedliche Vorzeichen, liegt die Nullstelle in $[a, m]$, sonst in $[m, b]$. Wiederholen, bis das Intervall kürzer als 10^{-8} ist.

1. Programmieren Sie das Verfahren mit einer `while`-Schleife.
2. Geben Sie das Ergebnis und die Anzahl der Schritte aus.

> **Erweiterung:** Wie viele Schritte erwarten Sie vorab? Begründen Sie.

Aufgabe 21 — Schleifen

Momentenverlauf

Für einen Einfeldträger unter Gleichstreckenlast gilt

$$M(x) = \frac{qx(L - x)}{2}$$

1. Klasse `SimpleBeam` mit `Length` und `Q` und Methode `Moment(double x)`.
2. Geben Sie mit einer `for`-Schleife eine Tabelle mit x und $M(x)$ an $n + 1$ gleichmäßig verteilten Stellen aus.
3. Ermitteln Sie in der Schleife das größte Moment und vergleichen Sie mit $qL^2/8$.

> **Erweiterung:** Was ändert sich, wenn n ungerade ist?

Aufgabe 22 — Arrays

Polynome

Ein Polynom $p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$ wird durch seine Koeffizienten beschrieben.

1. Klasse `Polynomial` mit Attribut `double[] Coefficients`.
2. Methode `Evaluate(double x)` wertet das Polynom aus.
3. Methode `Derivative()` gibt die Ableitung als neues `Polynomial` zurück.
4. Testen Sie mit $p(x) = 1 - 3x + 2x^2$.

> **Erweiterung:** Methode `Add(Polynomial other)` für Polynome mit unterschiedlichem Grad.

Aufgabe 23 — Vererbung

Knoten und Stäbe

1. Klasse `Point` mit `X`, `Y` und Methode `DistanceTo(Point other)`.
2. Klasse `Node`, die von `Point` erbt und zusätzlich eine Knotennummer hat. Rufen Sie den Konstruktor der Basisklasse mit `base` auf.
3. Klasse `Bar` mit Anfangs- und Endknoten und Methode `Length()`.
4. Erstellen Sie ein Dreiecksfachwerk und berechnen Sie die Gesamtlänge aller Stäbe.

> **Erweiterung:** Welche Variablentypen sind möglich: `Point p = new Node(...)` oder `Node n = new Point(...)`? Begründen Sie.

Aufgabe 24 — Listen und Schleifen

Bewehrung

Die Bewehrung eines Stahlbetonquerschnitts besteht aus mehreren Stabgruppen.

1. Klasse `RebarGroup` mit Anzahl n und Durchmesser d (mm) und Methode `Area()` in cm²: $A_s = n \cdot \pi d^2/4$.
2. Klasse `Reinforcement` mit einer Liste von Stabgruppen, Methoden `Add(RebarGroup g)` und `TotalArea()`.
3. Methode `IsSufficient(double requiredArea)`.
4. Testen Sie mit 4 Ø 16 + 2 Ø 12 gegen $A_{s,\mathrm{erf}} = 9.5$ cm².

> **Erweiterung:** Ermitteln Sie mit einer `while`-Schleife, wie viele Stäbe Ø 20 mindestens nötig sind, um $A_{s,\mathrm{erf}}$ allein abzudecken.

Aufgabe 25 Abstrakte Klassen

CO_2-Bilanz

Für ein Bauprojekt sollen die CO_2-Emissionen verschiedener Quellen bilanziert werden.

1. Abstrakte Klasse `EmissionSource` mit Attribut `Name` und abstrakter Methode `Co2()` in kg.
2. Unterklassen:
   - `Transport`: Strecke (km) × Emissionsfaktor (kg/km)
   - `Electricity`: Verbrauch (kWh) × 0.38 kg/kWh
   - `Heating`: Erdgas (kWh) × 0.20 kg/kWh
3. Klasse `Balance` mit Liste von Quellen und Methode `Total()`.

> **Erweiterung:** Geben Sie für jede Quelle ihren Anteil an der Gesamtemission in Prozent aus.

Aufgabe 26 Abstrakte Klassen

Regenabfluss

Der Abfluss einer Fläche ist $Q = \psi \cdot r \cdot A$ mit Abflussbeiwert ψ, Regenspende r in l/(s ha) und Fläche A in ha.

1. Abstrakte Klasse `Surface` mit Attribut `Area` (m²) und abstrakter Methode `RunoffCoefficient()`.
2. Unterklassen `Roof` ($\psi = 0.9$), `Asphalt` ($\psi = 0.9$) und `Lawn` ($\psi = 0.1$).
3. Klasse `Catchment` mit Liste von Flächen und Methode `Runoff(double r)` in l/s.
4. Testen Sie: Dach 400 m², Asphalt 250 m², Rasen 600 m², $r = 150$ l/(s ha).

> **Erweiterung:** Wie ändert sich der Abfluss, wenn der Asphalt durch Rasengittersteine ($\psi = 0.3$) ersetzt wird?

Aufgabe 27 Listen

Feinstaubmessung

Für PM10 gilt ein Tagesgrenzwert von 50 µg/m³, der höchstens 35-mal im Jahr überschritten werden darf.

1. Klasse `Station` mit `Name` und einer `List<double>` der Tagesmittelwerte.
2. Methode `ExceedanceDays(double limit)`: Anzahl der Tage über dem Grenzwert.
3. Methode `Complies()`: prüft, ob der Grenzwert eingehalten ist.
4. Methode `AnnualMean()`: Jahresmittelwert (Grenzwert 40 µg/m³).

> **Erweiterung:** Speichern Sie mehrere Stationen in einer Liste und geben Sie alle Stationen aus, die einen der Grenzwerte nicht einhalten.

Aufgabe 28 Schleifen

Lärmpegel addieren

Schallpegel addieren sich nicht linear, sondern energetisch:

$$L_{\text{ges}} = 10 \cdot \log_{10}\left(\sum_{i} 10^{L_i/10}\right)$$

1. Klasse `NoiseSource` mit `Name` und `Level` (dB, gemessen in 10 m Abstand).
2. Klasse `Site` mit Liste von Quellen und Methode `TotalLevel()`.
3. Testen Sie mit zwei Maschinen zu 70 dB und einer zu 75 dB.

> **Erweiterung:** Der Pegel nimmt mit dem Abstand ab: $L(s) = L_{\text{ges}} - 20\log_{10}(s/10\ \text{m})$. Ab welchem Abstand (in 1-m-Schritten) wird 55 dB unterschritten?

Aufgabe 29 Schleifen

Schadstoffabbau

In einem Gewässer wird ein Schadstoff mit der Rate k abgebaut. Für einen kleinen Zeitschritt Δt gilt näherungsweise

$$c(t + \Delta t) = c(t) \cdot (1 - k\Delta t)$$

1. Simulieren Sie mit einer `while`-Schleife den Verlauf für $c_0 = 20$ mg/l, $k = 0.23$ 1/d und $\Delta t = 0.1$ d.
2. Nach wie vielen Tagen ist $c < 5$ mg/l?
3. Vergleichen Sie mit der exakten Lösung $c(t) = c_0 e^{-kt}$.

> **Erweiterung:** Wie verändert sich das Ergebnis mit $\Delta t = 1$ d und $\Delta t = 0.01$ d?

Aufgabe 30 Dictionary und HashSet

Abfallbilanz

Auf einem Wertstoffhof werden Anlieferungen mit Abfallart und Masse (t) erfasst.

1. Klasse `Delivery` mit `Fraction` (z.B. „Papier“, „Glas“, „Restabfall“) und `Mass`.
2. Klasse `WasteBalance` mit Liste von Anlieferungen.
3. Methode `MassByFraction()`: Masse je Abfallart als `Dictionary<string, double>`.
4. Methode `RecyclingRate()`: Anteil der verwertbaren Abfallarten an der Gesamtmasse. Die verwertbaren Arten stehen in einem `HashSet<string>`.